Classificação de imagens com redes neurais convolucionais

Intel Image Classification — CNN treinada do zero

Guilherme Grigolin da Mata Brandão — Bacharelado em Sistemas de Informação, IFSP

Outubro de 2026

1. Objetivo e referência utilizada

O objetivo foi treinar do zero (pesos aleatórios) uma CNN que classifica cenas em seis classes — *buildings*, *forest*, *glacier*, *mountain*, *sea* e *street* — e medir, com experimentos controlados, o efeito da taxa de aprendizado, da capacidade da rede e de técnicas de regularização. O notebook de referência foi *Intel Image Classification (CNN - Keras)*, de Vincenzooo (Kaggle, usuário *vincee*), acessado em 27/10/2026. Dele foram aproveitados o uso do teste oficial, a normalização para [0, 1], a CNN Keras com blocos Conv2D/ReLU/MaxPooling, a saída softmax com *sparse categorical crossentropy* e Adam, as curvas de treino e a matriz de confusão. Foram adaptados: validação estratificada e agrupada por duplicatas (a referência usa *validation_split* sem estratificação), entrada 128×128 com GlobalAveragePooling em vez de Flatten, EarlyStopping, checkpoints, F1 macro e o conjunto de oito experimentos. A parte de transferência de aprendizado (VGG16) da referência não foi usada. Implementação em Python 3.13 com TensorFlow/Keras 2.21/3.13, executada no Google Colab com GPU (T4); versões completas em *resultados/ambiente.json*. Semente: 42.

2. Preparação dos dados

A versão baixada (via *kagglehub*) contém *seg_train* (14.034 imagens), *seg_test* (3.000) e *seg_pred* (sem rótulo, descartada por não permitir medir acurácia ou F1). Todos os 17.034 arquivos rotulados foram decodificados com Pillow: **nenhum corrompido**; 16.979 têm 150×150 e 55 têm altura menor. Para duplicatas calculou-se o MD5 (cópias exatas) e o *perceptual hash* (pHash) de cada imagem; pares com MD5 igual ou distância de Hamming ≤ 6 foram unidos em grupos (limiar escolhido por inspeção visual: até 6 os pares eram a mesma foto; a partir de 8 surgiam cenas diferentes). Foram achados 96 grupos com 209 imagens. Regras aplicadas: 42 imagens do treino que repetiam imagens do teste foram excluídas (o teste foi preservado) e 20 imagens do treino em grupos com rótulos conflitantes foram excluídas por ambiguidade (lista em *exclusoes.csv*).

Como há divisão oficial, o teste foi mantido e ~20% do treino foi separado para validação com *StratifiedGroupKFold* (5 dobras, primeira dobra, semente 42): estratificado por classe e com cada grupo de duplicatas inteiro em um único conjunto. Verificou-se por código que nenhum grupo aparece em dois conjuntos. A mesma divisão foi usada em todos os experimentos e as listas de arquivos estão em *split_treino/validacao/teste.csv*. As imagens foram redimensionadas para 128×128 (bilinear) e os pixels divididos por 255. O aumento de dados, quando usado, foi aplicado só ao treino, após a divisão.

Classe	Treino	Validação	Teste
buildings	1747	438	437
forest	1806	451	474
glacier	1904	476	553
mountain	2004	498	525
sea	1816	455	510
street	1903	474	501
TOTAL	11180	2792	3000

Tabela 1 — Imagens por classe e conjunto, após as exclusões.

3. Modelo inicial

A CNN inicial (E0) segue a configuração sugerida: entrada RGB 128×128×3; três blocos Conv2D 3×3 (*padding same*, ReLU) com 32, 64 e 128 filtros, cada um seguido de MaxPooling 2×2 (saídas 64×64×32, 32×32×64 e 16×16×128); GlobalAveragePooling (vetor de 128); Dense(128, ReLU); Dense(6, softmax). Total de **110.534 parâmetros treináveis**, inicializados aleatoriamente (Glorot). As convoluções extraem padrões locais cada vez mais abstratos (bordas → texturas → partes da cena), o pooling reduz a resolução e dá tolerância a deslocamentos, o GAP resume cada mapa de ativação por sua média e as camadas densas combinam essas características. A softmax produz seis probabilidades que somam 1, e a classe prevista é a de maior probabilidade (*argmax*). Como os rótulos são inteiros 0–5, a perda compatível é a *sparse categorical crossentropy*. Treino: Adam (lr = 0,001), mini-lotes de 32, até 30 épocas, EarlyStopping na perda de validação (paciência 5, restaurando os melhores pesos) e ModelCheckpoint do menor *val_loss*. Cada experimento reinicia a semente e cria um modelo novo. As métricas da Tabela 2 foram calculadas após recarregar o checkpoint, em modo de avaliação e sem aumento de dados; o teste não foi consultado nesta fase.

4. Experimentos

E1–E6 alteram um único fator em relação ao E0. E2 adiciona um quarto bloco Conv2D(256) (mais profundidade e campo receptivo). E3/E4 inserem Dropout 0,3/0,5 entre a Dense(128) e a saída. E5 aplica L2 = 0,0001 aos *kernels* das três Conv2D e da Dense(128). E6 usa espelhamento horizontal, rotação de até ±11° e zoom de até ±10%. A regra do E7 foi fixada antes dos resultados: combinar as alterações que superaram o E0 no F1 macro de validação (para Dropout, a melhor taxa) — resultaram E2 + E3 + E6. No E5 a perda do Keras inclui o termo L2; por isso a tabela traz também a entropia cruzada pura (CE), que é a comparável entre experimentos.

ID	Alteração	Parâm.	Épocas (melhor)	Tempo (s)	Acc treino	Acc val	CE treino	CE val	F1 macro val
E0	Modelo inicial	110.534	30 (30)	244	0.890	0.869	0.288	0.386	0.8695
E1	lr = 0,0001	110.534	30 (30)	238	0.729	0.726	0.748	0.739	0.7289
E2	4 blocos (…256)	422.086	23 (18)	203	0.930	0.881	0.198	0.357	0.8822
E3	Dropout 0,3	110.534	30 (30)	235	0.897	0.879	0.281	0.376	0.8804
E4	Dropout 0,5	110.534	30 (30)	235	0.893	0.867	0.296	0.387	0.8676
E5	L2 0,0001 (perda total val 0,476)	110.534	30 (27)	235	0.870	0.854	0.363	0.410	0.8538
E6	Aumento de dados	110.534	30 (30)	954	0.878	0.874	0.327	0.379	0.8747
E7	E2 + E3 + E6	422.086	30 (25)	1006	0.904	0.883	0.261	0.350	0.8835

Tabela 2 — Resultados de validação (checkpoint de menor val_loss; lr 0,001, batch 32 e semente 42, salvo indicação). Tempos medidos na GPU do Colab.

Leitura das curvas. (Figura 1) No E0 a perda de validação ainda caía na 30ª época (melhor época = 30) e as acurácias de treino e validação ficaram próximas (0,890 × 0,869 na avaliação final): não houve overfitting; o modelo estava levemente **subtreinado**, limitado pelo teto de 30 épocas. Com lr = 0,0001 (E1) a convergência ficou muito lenta: 0,73 de acurácia após 30 épocas, **underfitting** claro. A rede maior (E2) convergiu mais rápido (melhor época 18) e, depois disso, a perda de treino continuou caindo enquanto a de validação subiu (0,357 → 0,388), início de **overfitting** que o EarlyStopping interrompeu na época 23; foi o maior gap treino-validação (0,930 × 0,881). Dropout 0,3 deu um pequeno ganho (+0,011 de F1); 0,5 ficou um pouco abaixo do E0. L2 piorou o F1 (−0,016): a penalidade restringe pesos de uma rede que não estava sobreajustando. O aumento de dados elevou o F1 em 0,005 e manteve a acurácia de validação acima da de treino, mas custou ~4× o tempo (a transformação roda na CPU do Colab).

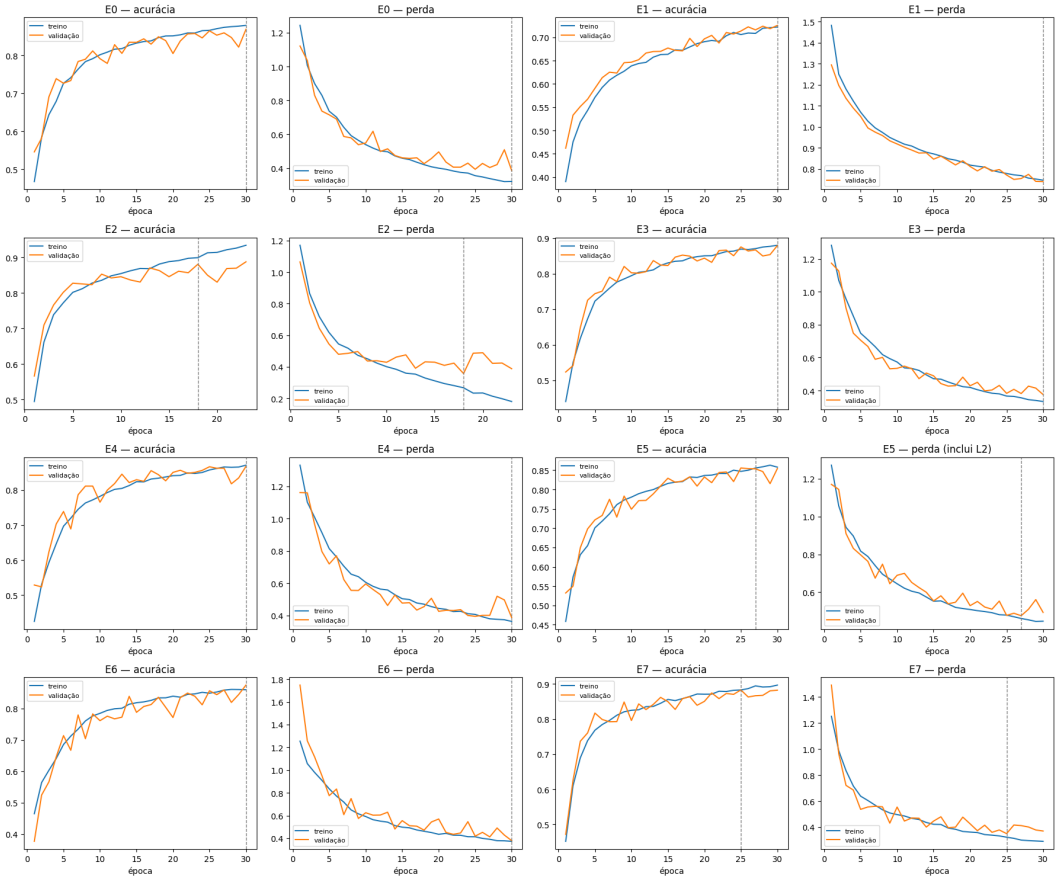


Figura 1 — Acurácia e perda de treino e validação de E0–E7 (linha tracejada: melhor época). As curvas de treino são registradas com Dropout/aumento ativos; no E5 a perda inclui L2.

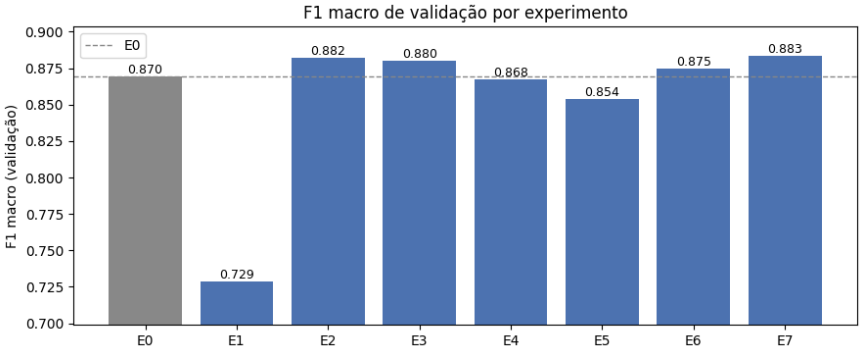


Figura 2 — F1 macro de validação por experimento.

5. Avaliação final

Pelo critério de maior F1 macro de validação, foi selecionado o **E7** (0,8835), à frente do E2 (0,8822) por apenas 0,0013 — diferença acima do limiar de empate adotado (0,001), mas dentro da variação esperada entre sementes. Após encerrar os ajustes, o checkpoint do E7 (época 25) foi avaliado uma única vez no teste: **acurácia 0,8737 e F1 macro 0,8748**, próximos da validação (sem sinal de sobreajuste à validação).

Classe	Precisão	Recall	F1	Suporte
buildings	0.8132	0.8764	0.8436	437
forest	0.9848	0.9557	0.9700	474
glacier	0.8367	0.8336	0.8351	553
mountain	0.8583	0.7848	0.8199	525
sea	0.8800	0.9059	0.8928	510
street	0.8772	0.8982	0.8876	501
macro	0.8750	0.8758	0.8748	3000
acurácia			0.8737	3000

Tabela 3 — Métricas no teste (modelo E7).

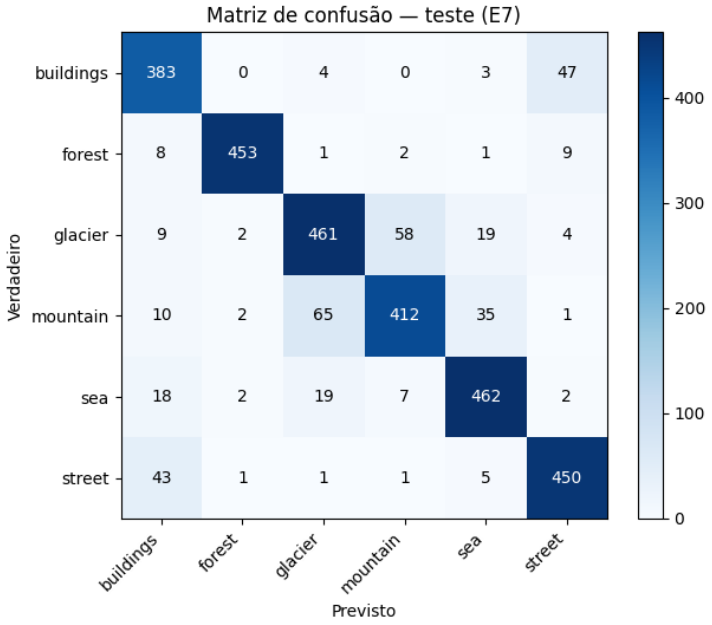


Figura 3 — Matriz de confusão no teste.

As confusões mais frequentes foram **mountain ↔ glacier** (65 + 58), **buildings ↔ street** (47 + 43) e mountain → sea (35). São pares que compartilham conteúdo visual: montanhas nevadas e geleiras têm as mesmas cores (branco/azul) e relevo; ruas quase sempre contêm prédios e fotos de prédios mostram a rua; e montanhas com lago ou horizonte azul lembram o mar. *Forest* foi a classe mais fácil (F1 0,970), pela textura verde muito característica. A Figura 4 mostra exemplos: vários erros têm rótulo discutível (uma rua com fachada em destaque, uma montanha coberta de neve).

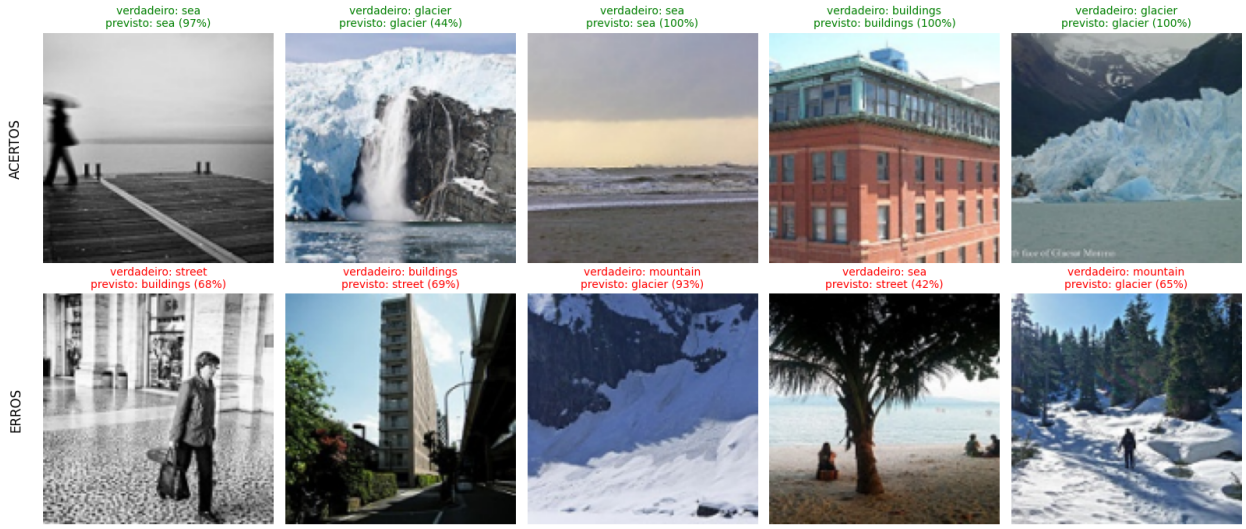


Figura 4 — Cinco acertos (acima) e cinco erros (abaixo) no teste, com classe verdadeira, prevista e confiança.

6. Conclusões

- **Overfitting ou underfitting no E0?** Nenhum overfitting: as curvas de validação acompanharam as de treino e a perda de validação ainda caía na última época. Houve leve underfitting por falta de épocas.
- **Taxa de aprendizado e capacidade.** lr = 0,0001 tornou o treino lento demais para 30 épocas (F1 0,729). Mais um bloco (E2) acelerou a convergência e deu o melhor resultado isolado (0,882), mas quadruplicou os parâmetros (110 mil → 422 mil) e passou a sobreajustar após a época 18.
- **Regularização.** Dropout 0,3 trouxe o maior benefício individual (+0,011), seguido do aumento de dados (+0,005). L2 (−0,016) e Dropout 0,5 (−0,002) pioraram, coerente com um modelo base que não sobreajustava.
- **Classes mais confundidas.** mountain ↔ glacier e buildings ↔ street, por cores, texturas e objetos em comum.

- **A configuração mais complexa foi a melhor?** Sim, o E7 venceu, mas por margem mínima sobre o E2 (+0,0013 de F1), com os mesmos 422 mil parâmetros e ~5× o tempo de treino (1006 s × 203 s). O ganho não compensa claramente o custo.
- **Limitações e próximo passo.** Uma única semente (diferenças de ~0,01 podem ser ruído); teto de 30 épocas que interrompeu modelos ainda melhorando; rótulos ruidosos na base. O próximo experimento seria repetir E0, E2 e E7 com 3–5 sementes e até 60 épocas, testar BatchNormalization e agendamento da taxa de aprendizado e, como comparação extra, transferência de aprendizado.

Referências

VINCENZOOO (usuário vincee). *Intel Image Classification (CNN - Keras)*. Kaggle, [s.d.]. Disponível em: https://www.kaggle.com/code/vincee/intel-image-classification-cnn-keras. Acesso em: 27 out. 2026.

BANSAL, Puneet. *Intel Image Classification* (dataset). Kaggle, 2019. Disponível em: https://www.kaggle.com/datasets/puneet6060/intel-image-classification. Acesso em: 27 out. 2026.

CHOLLET, François. *Deep Learning with Python*. 2. ed. Manning, 2021.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016.

Documentação do TensorFlow/Keras e do scikit-learn (StratifiedGroupKFold, métricas). Notebook-base da disciplina (*intel_image_classification.ipynb*).

Uso de IA: foi utilizado o assistente Claude (Anthropic), via Claude Code, como apoio na escrita do código do notebook, na organização dos experimentos e na redação deste relatório e do questionário. Todos os resultados vêm da execução real do notebook no Google Colab.